

HM ==> not in insertion order

LHM ==> insertion order

TM ==> sorting based on the keys

Array with 5 elements

[10, 40, 3, 56, 7]

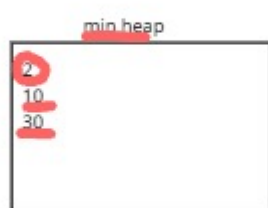
Array of Data Stream(data will keep coming)

[10, 2, 30,]

smallest

**Min Heap ==> you have a list with
10 elements OR 1000 ele OR 1,00,000..... ==> always
going to give you the smallest element**

[10, 2, 30,]



If we want to use Min Heap concept in Java, then java provided us with a built in class ==> PriorityQueue

```
new PriorityQueue();
```

minHeap ==>

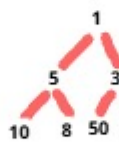


min heap

heap(a way of arranging/storing the elements in tree format)

parent < child

```
minHeap.add(5);
minHeap.add(3);
minHeap.add(50);
minHeap.add(10);
minHeap.add(8);
minHeap.add(1);
```



[1, 5, 3, 10, 8, 50]

**Max Heap ==> always going to give you
largest element**

condition / rule ==> parent > child

**pick the 2 heavy weight stones and smash(-)
then**

12 - 10 ==> 2

stones = [8, 3, 4, 6, 2]

stones = [3, 4, 2, 2]

stones = [1, 2, 2]

stones = [1]

```
while(maxHeap.size() > 1)
```

```
{
```

```
}
```

stones = [10, 8, 3, 4, 6, 12]

Find the median in a data stream

median ==> middle value after sorting

arr = [10, 3, 4, 12]

[3, 4, 10, 12]

X

[2, 3, 6, 8, 11]

max min
[2, 3] [8, 11]

max ==> smaller ele

min ==> larger ele

[10, 3, 4, 12]

